AutoMail Pro

Technical Documentation and Stack Justification

Email Automation MVP | Prepared by Sai Kishan

1. Project Objective

AutoMail Pro is a simple email automation solution designed to help users prepare, personalize, preview, and send emails to multiple recipients from a CSV or Excel file. The objective is to reduce repetitive manual email work while maintaining control, traceability, and basic error handling.

For real-time testing, the solution was configured using a personal project email account. This avoids using institutional or real user data during development and demonstrates that the sending workflow works with a real SMTP provider.

2. Scope of the MVP

Feature	Description
Recipient import	Upload recipient data from CSV or Excel files.
Validation	Check required columns, missing fields, and invalid email formats.
Template personalization	Replace variables such as {first_name}, {last_name}, {company_name}, and {email}.
Preview	Allow the user to review generated emails before sending.
SMTP sending	Send emails using configured SMTP credentials.
Status tracking	Track each recipient as pending, sent, or failed.
Campaign history	Store campaign records and email logs in SQLite.
Dashboard	Display basic totals for campaigns, sent emails, and failed emails.

3. Technology Stack and Reasons for Selection

Technology	Role	Reason for Choosing
Python	Main backend language	Chosen because it is simple, readable, fast to develop with, and has strong support for automation, data processing, file handling, and SMTP email sending. It is also suitable for a student/freelance MVP because the code is easy to explain and maintain.
Streamlit	User interface	Chosen because it allows quick development of a clean web interface without building a separate frontend using HTML, CSS, or JavaScript. It is ideal for demos, internal tools, and MVPs where speed and simplicity are important.
Pandas	CSV/Excel processing	Chosen because it provides reliable and simple handling of tabular data. It makes it easy to load, validate, clean, display, and process recipient lists from CSV or Excel files.
openpyxl	Excel file support	Chosen because Pandas uses it to read .xlsx files. It enables the app to support Excel imports, which are common for non-technical users.
SMTP / smtplib	Email delivery	Chosen because SMTP is a standard protocol supported by Gmail, Outlook, SendGrid, and many other providers. Python includes smtplib by default, so the sending logic remains lightweight and does not require a paid email API.
SQLite	Local database	Chosen because it is lightweight, file-based, and requires no separate database server. It is perfect for storing campaign history and email logs in an MVP.
python-dotenv	Configuration management	Chosen to keep sensitive SMTP credentials outside the source code. Credentials are stored locally in a .env file instead of being hardcoded.

4. High-Level Architecture

The application follows a simple modular architecture. The Streamlit app acts as the user-facing layer, while separate Python modules handle validation, template processing, email sending, and database operations.

```
User Interface (Streamlit app.py)
    |
    +-- Recipient Loader (CSV/Excel import)
    +-- Validator (required fields + email format)
    +-- Template Engine (placeholder replacement)
    +-- Email Sender (SMTP + STARTTLS)
    +-- Database Layer (SQLite campaign history + logs)
```

5. Code Structure

File / Folder	Purpose
app.py	Main Streamlit application. Contains the interface, workflow, session state, preview, sending process, and dashboard display.
src/recipient_loader.py	Loads CSV or Excel recipient files using Pandas and adds initial validation status for each row.
src/validator.py	Validates required columns, missing fields, and email address format using a regular expression.
src/template_engine.py	Replaces template placeholders with recipient-specific values.
src/email_sender.py	Handles actual SMTP email sending using smtplib, MIME messages, STARTTLS, login, and error handling.
src/database.py	Creates and manages SQLite tables for campaigns and email logs.
src/utils.py	Loads environment variables and checks whether SMTP configuration is complete.
data/sample_recipients.csv	Sample fake recipient data for testing.
.env.example	Template file showing required SMTP configuration variables.
requirements.txt	Python dependencies required to run the project.

6. Email Sending Flow

1. The user uploads a CSV or Excel file containing recipient data.

2. The system checks that required columns are present: first_name, last_name, company_name, and email.

3. Each row is validated for missing values and valid email format.

4. The user writes a subject and body using placeholders such as {first_name}.

5. The system generates a personalized preview for each valid recipient.

6. When the user sends the campaign, each email is sent using the configured SMTP provider.

7. Each send attempt is logged as sent or failed, with the error message stored when failure occurs.

8. Campaign history and summary statistics are saved in SQLite.

7. SMTP Configuration

SMTP settings are stored in a local .env file. This is safer than putting credentials directly inside the Python source code.

```
SMTP_SERVER=smtp.office365.com
SMTP_PORT=587
EMAIL_ADDRESS=your_project_email@outlook.com
EMAIL_PASSWORD=your_password_or_app_password
SENDER_NAME=Sai Kishan
```

For real-time testing, a personal project email account was used. This allowed the sending feature to be tested without using real personal data or institutional credentials.

8. Why SMTP Was Used

SMTP was selected because it is provider-independent and simple for an MVP. The same code structure can work with Outlook, Gmail, SendGrid, or other SMTP providers by changing the configuration values in the .env file. This keeps the project flexible and avoids locking the solution to one vendor.

The email sender uses STARTTLS before login. STARTTLS upgrades the SMTP connection to an encrypted connection, which is required by many modern email providers when sending authenticated emails.

9. Error Handling

Error Type	Handling Method
Missing required columns	The app stops the import and explains which columns are missing.
Missing recipient information	The row is marked as failed with a specific missing-field message.
Invalid email address	The row is marked as failed before attempting to send.
SMTP authentication failure	The system returns a clear message asking the user to check email/app password.
Recipient refused	The system marks the email as failed and logs the refusal.
Generic SMTP error	The system captures the SMTP exception and stores it in the log.

10. Data Privacy and Safety

- No real personal data is required for testing. Fake recipient data should be used during demos.
- SMTP credentials are stored in .env and should never be committed to GitHub.
- The app supports preview before sending, reducing the risk of accidental or incorrect emails.
- A personal project email account was used for testing instead of a primary personal, university, or production account.
- The local SQLite database stores campaign logs only for demonstration and MVP tracking.

11. Demonstration Plan

1. Start the app using streamlit run app.py.

2. Upload the sample CSV or Excel recipient file with fake recipients.

3. Write a subject using placeholders, for example: Hello {first_name} from {company_name}.

4. Write an email body using the available variables.

5. Preview the personalized emails before sending.

6. Run the campaign in demo mode first to show the logic safely.

7. Run one real test email using the personal project email account to prove SMTP delivery works.

8. Show the campaign history and sent/failed statistics.

12. Possible Improvements

Improvement	Description
Scheduling	Allow users to schedule campaigns for a future date and time.
Authentication	Add login so multiple users can use the tool securely.
Template library	Save reusable templates for future campaigns.
HTML emails	Support styled HTML templates instead of plain text only.
Attachments	Allow users to attach PDFs or documents.
SendGrid API integration	Use a dedicated email service for better delivery, statistics, and scalability.
Unsubscribe management	Required for newsletter or marketing-style emails.
Open/click tracking	Track engagement using email service provider features.

13. Short Technical Explanation

The solution was intentionally designed as a simple MVP rather than a complex production platform. Python and Streamlit reduce development time, Pandas simplifies recipient import, SMTP provides provider-independent sending, and SQLite gives lightweight campaign logging. The project demonstrates the complete automation logic: import data, validate recipients, personalize templates, preview emails, send through SMTP, and track results.